claire_model

Load data

```
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.4.3
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   4.0.2     v tibble    3.2.1
v lubridate 1.9.3     v tidyr     1.3.1
v purrr     1.0.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
library(mgcv)
```

```
Loading required package: nlme
```

```
Attaching package: 'nlme'
```

```
The following object is masked from 'package:dplyr':

    collapse
```

```
This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
```

```
train <- read_csv("data-train.csv")
```

```
Rows: 89 Columns: 7
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (7): St, Re, Fr, R_moment_1, R_moment_2, R_moment_3, R_moment_4

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
train <- train |>
  mutate(
    m1 = R_moment_1, m2 = R_moment_2,
    m3 = R_moment_3, m4 = R_moment_4,

    # Central moments
    C_moment_2 = m2 - m1^2,
    C_moment_3 = m3 - 3*m1*m2 + 2*m1^3,
    C_moment_4 = m4 - 4*m1*m3 + 6*m1^2*m2 - 3*m1^4,

    # Summary statistics
    sd_R = sqrt(C_moment_2),
    skewness = C_moment_3 / sd_R^3,
    kurtosis = C_moment_4 / sd_R^4,
    ex_kurt = kurtosis - 3,

    # Keep infinite Fr as a valid category
    Fr = as.factor(Fr),
    Re = as.factor(Re)
  )
```

Mean spline model

```
model_mean <- gam(
  log(R_moment_1) ~ Re + Fr + s(St, k = 5),
  data = train,
  method = "REML"
)
```

```r
summary(model_mean)
```

```
Family: gaussian
Link function: identity

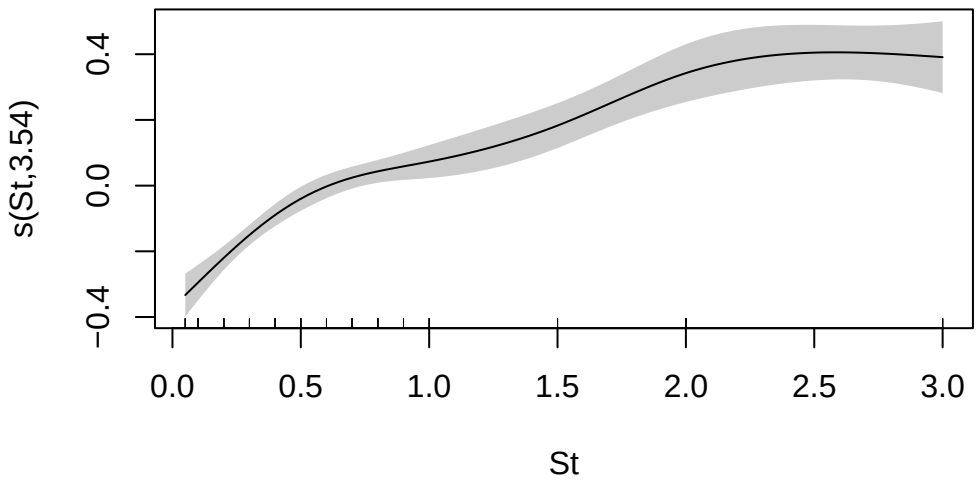
Formula:
log(R_moment_1) ~ Re + Fr + s(St, k = 5)

Parametric coefficients:
            Estimate Std. Error  t value Pr(>|t|)
(Intercept) -2.18389    0.03144  -69.466  < 2e-16 ***
Re224       -3.63949    0.03363 -108.213  < 2e-16 ***
Re398       -5.77972    0.04254 -135.864  < 2e-16 ***
Fr0.3       -0.11795    0.03871   -3.047  0.00313 **
FrInf       -0.02276    0.03450   -0.660  0.51138
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

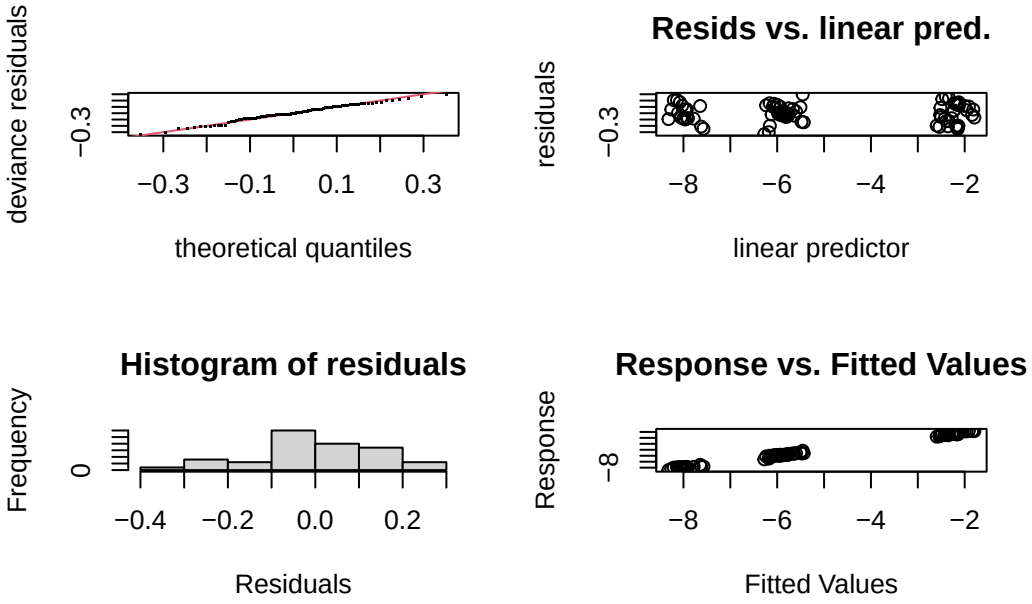
Approximate significance of smooth terms:
        edf Ref.df     F p-value
s(St) 3.537  3.859 50.11  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.996   Deviance explained = 99.6%
-REML = -31.975  Scale est. = 0.019356  n = 89
```

```r
plot(model_mean, shade = TRUE)
```




```
gam.check(model_mean)
```



```
Method: REML   Optimizer: outer newton
full convergence after 5 iterations.
Gradient range [-6.099059e-06,2.025272e-07]
(score -31.97488 & scale 0.01935551).
Hessian positive definite, eigenvalue range [0.5642027,41.53933].
Model rank =  9 / 9

Basis dimension (k) checking results. Low p-value (k-index<1) may
indicate that k is too low, especially if edf is close to k'.

        k'  edf k-index p-value
s(St) 4.00 3.54    0.79    0.01 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Standard deviation spline model

```r
model_sd <- gam(
  log(sd_R) ~ Re + Fr + s(St, k = 5),
  data = train,
  method = "REML"
)
```

```r
summary(model_sd)
```

```
Family: gaussian
Link function: identity

Formula:
log(sd_R) ~ Re + Fr + s(St, k = 5)

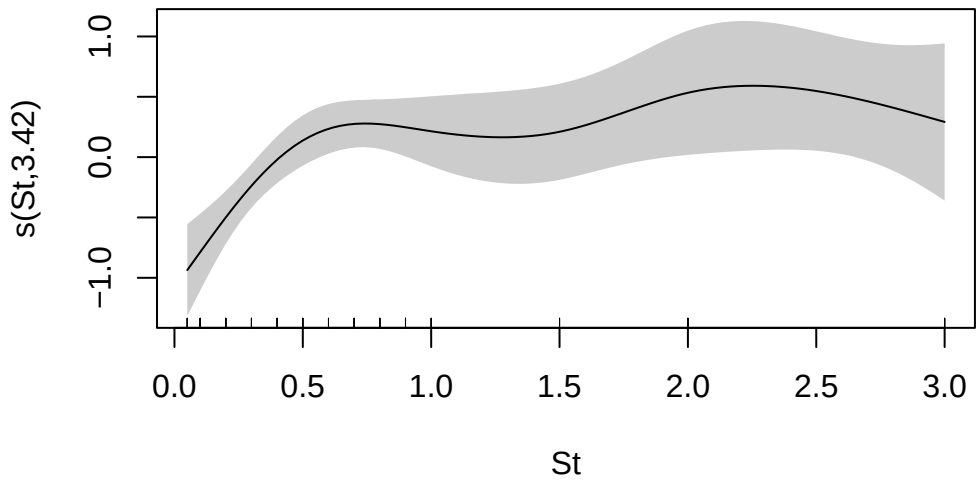
Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   1.9579     0.1879  10.420  < 2e-16 ***
Re224        -2.3652     0.2010 -11.767  < 2e-16 ***
Re398        -3.9885     0.2541 -15.699  < 2e-16 ***
Fr0.3        -1.7622     0.2314  -7.615  4.4e-11 ***
FrInf        -1.3763     0.2062  -6.674  2.9e-09 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

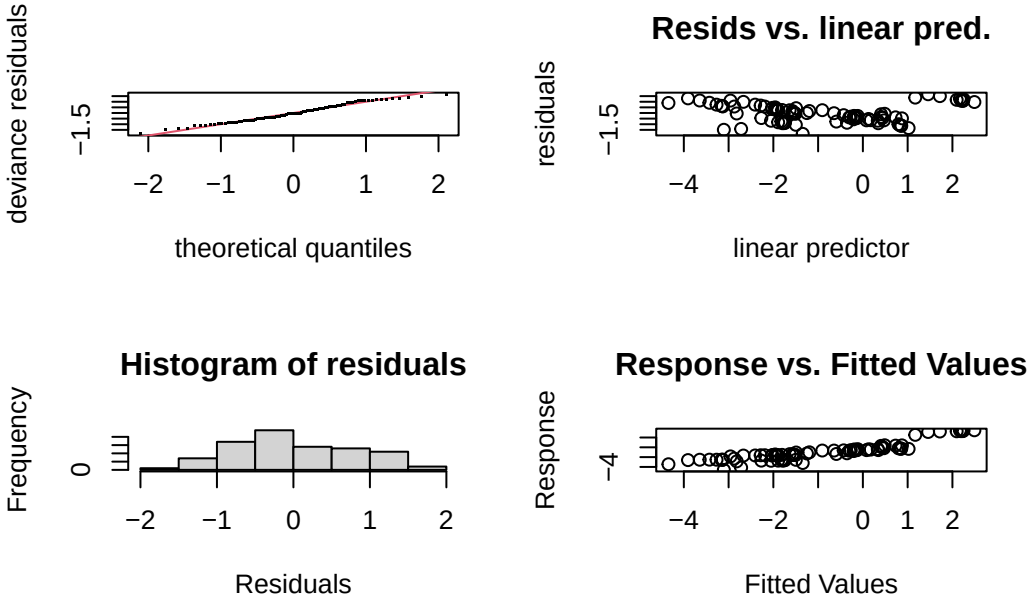
Approximate significance of smooth terms:
        edf Ref.df     F  p-value
s(St) 3.423  3.787 5.968 0.000275 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.799   Deviance explained = 81.6%
-REML = 116.01  Scale est. = 0.69155   n = 89
```

```r
plot(model_sd, shade = TRUE)
```




```r
gam.check(model_sd)
```




```
Method: REML   Optimizer: outer newton
full convergence after 4 iterations.
Gradient range [-7.049749e-06,-4.32155e-06]
(score 116.0126 & scale 0.6915505).
Hessian positive definite, eigenvalue range [0.4244224,41.53573].
Model rank =  9 / 9

Basis dimension (k) checking results. Low p-value (k-index<1) may
indicate that k is too low, especially if edf is close to k'.

        k'  edf k-index p-value
s(St) 4.00 3.42    0.93    0.28
```

Skewness spline model

```r
model_skewness <- gam(
  log(skewness) ~ Re + Fr + s(St, k = 5),
  data = train,
  method = "REML"
)
```

```r
summary(model_skewness)
```

```
Family: gaussian
Link function: identity

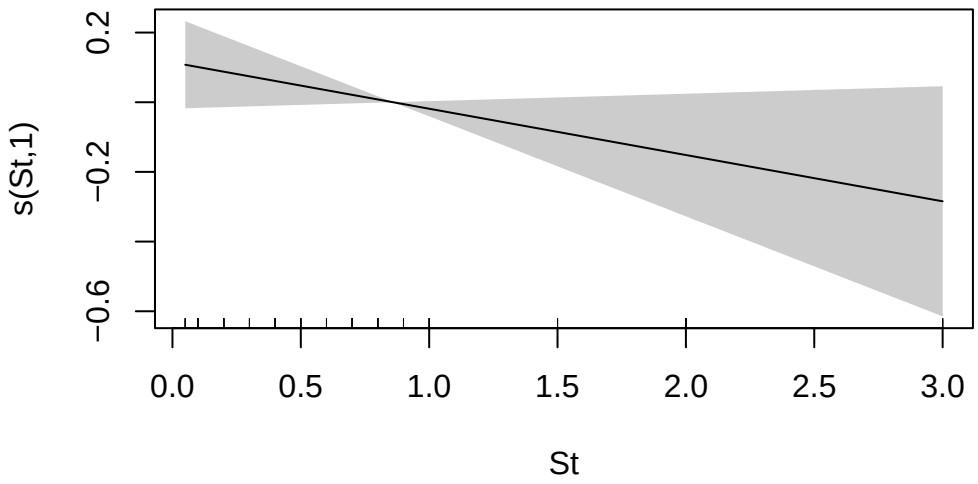
Formula:
log(skewness) ~ Re + Fr + s(St, k = 5)

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   4.7427     0.1278  37.111  < 2e-16 ***
Re224         1.0671     0.1367   7.807 1.57e-11 ***
Re398         1.4694     0.1719   8.546 5.28e-13 ***
Fr0.3        -1.6423     0.1576 -10.423  < 2e-16 ***
FrInf        -1.3365     0.1406  -9.509 6.27e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

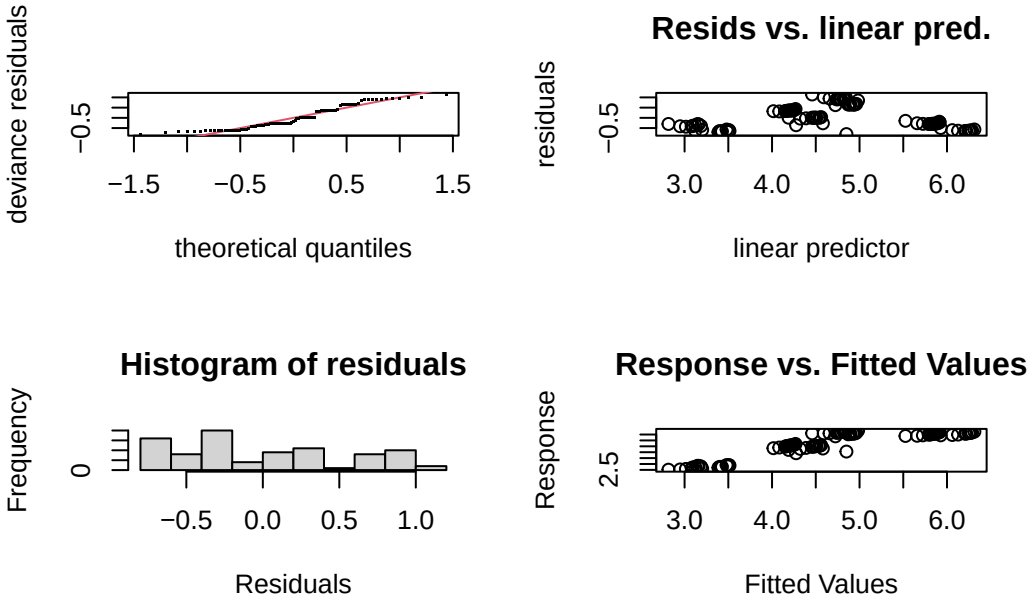
Approximate significance of smooth terms:
      edf Ref.df     F p-value
s(St)   1      1 2.967  0.0887 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.748   Deviance explained = 76.3%
-REML = 80.771  Scale est. = 0.32199   n = 89
```

```r
plot(model_skewness, shade = TRUE)
```




```
gam.check(model_skewness)
```



```
Method: REML   Optimizer: outer newton
full convergence after 8 iterations.
Gradient range [-4.525514e-05,1.140528e-05]
(score 80.77056 & scale 0.3219863).
Hessian positive definite, eigenvalue range [4.525215e-05,41.49999].
Model rank =  9 / 9

Basis dimension (k) checking results. Low p-value (k-index<1) may
indicate that k is too low, especially if edf is close to k'.

      k' edf k-index p-value
s(St)  4   1    1.06    0.67
```

Kurtosis spline model

```r
model_kurtosis <- gam(
  log(kurtosis) ~ Re + Fr + s(St, k = 5),
  data = train,
  method = "REML"
)
```

```r
summary(model_kurtosis)
```

```
Family: gaussian
Link function: identity

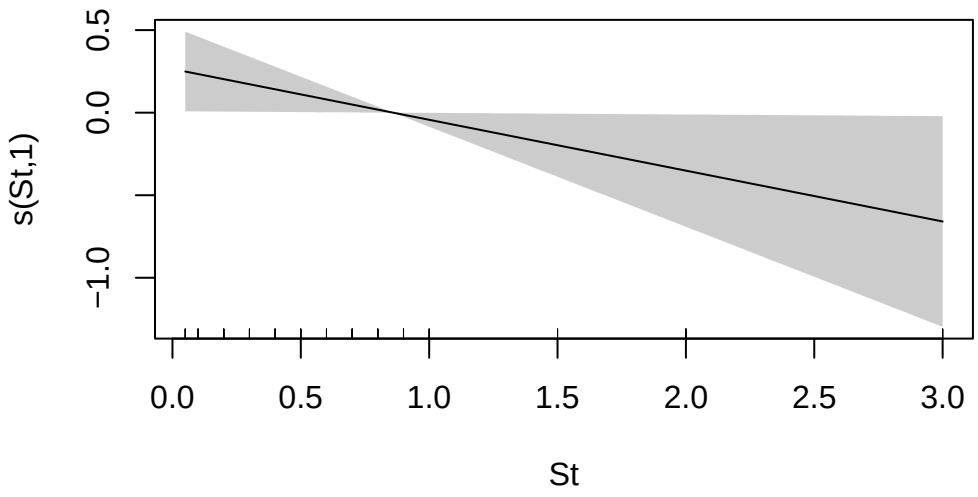
Formula:
log(kurtosis) ~ Re + Fr + s(St, k = 5)

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   9.5622     0.2469  38.735  < 2e-16 ***
Re224         2.0730     0.2640   7.852 1.28e-11 ***
Re398         2.8498     0.3321   8.580 4.51e-13 ***
Fr0.3        -3.2577     0.3044 -10.704  < 2e-16 ***
FrInf        -2.6467     0.2715  -9.749 2.08e-15 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

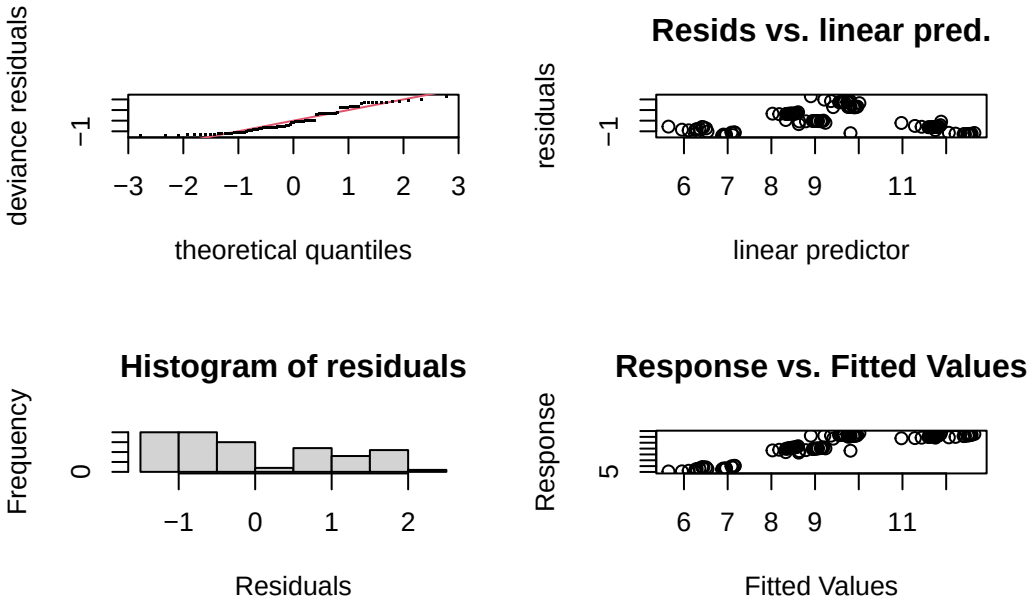
```
Approximate significance of smooth terms:
       edf Ref.df     F p-value
s(St)    1      1 4.268  0.0419 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.756   Deviance explained = 76.9%
-REML = 135.42  Scale est. = 1.2014    n = 89
```

```r
plot(model_kurtosis, shade = TRUE)
```




```r
gam.check(model_kurtosis)
```




```
Method: REML   Optimizer: outer newton
full convergence after 8 iterations.
Gradient range [-5.780349e-05,2.576094e-06]
(score 135.4156 & scale 1.201417).
Hessian positive definite, eigenvalue range [5.779822e-05,41.5].
Model rank =  9 / 9

Basis dimension (k) checking results. Low p-value (k-index<1) may
indicate that k is too low, especially if edf is close to k'.

      k' edf k-index p-value
s(St)  4   1    1.07    0.71
```

70/30 split evaluation

```
set.seed(2026)
data_train <- train |>
  slice_sample(prop = 0.7)
data_test <- train |>
  anti_join(data_train, by = c("St", "Re", "Fr"))
```

```r
# Refit the same four model formulas on the training portion only.
model_mean_train <- update(model_mean, data = data_train)
model_sd_train <- update(model_sd, data = data_train)
model_skewness_train <- update(model_skewness, data = data_train)
model_kurtosis_train <- update(model_kurtosis, data = data_train)
```

```r
predict_original <- function(model, newdata) {
  smearing_factor <- mean(exp(residuals(model, type = "response")))
  exp(predict(model, newdata = newdata)) * smearing_factor
}

predictions <- data_test |>
  mutate(
    pred_mean = predict_original(model_mean_train, data_test),
    pred_sd = predict_original(model_sd_train, data_test),
    pred_skewness = predict_original(model_skewness_train, data_test),
    pred_kurtosis = predict_original(model_kurtosis_train, data_test)
  )
```

```r
score_predictions <- function(response, observed, predicted) {
  tibble(
    Response = response,
    RMSE = sqrt(mean((observed - predicted)^2)),
    MAE = mean(abs(observed - predicted))
  )
}

performance <- bind_rows(
  score_predictions("Mean", predictions$R_moment_1, predictions$pred_mean),
  score_predictions("Standard deviation", predictions$sd_R, predictions$pred_sd),
  score_predictions("Skewness", predictions$skewness, predictions$pred_skewness),
  score_predictions("Kurtosis", predictions$kurtosis, predictions$pred_kurtosis)
)
performance
```

```
# A tibble: 4 x 3
  Response                RMSE         MAE
  <chr>                  <dbl>       <dbl>
1 Mean                  0.0122     0.00673
2 Standard deviation    7.64       3.30
3 Skewness            131.        97.9
```

```
4 Kurtosis            129247.     74505.
```